RED HAT ENTERPRISE LINUX

LINUX ADMINISTRATION PRACTICALS 1–6

Commands in bold with a one-line description for every command

Prepared in the same practical order as the college index

- 1. Installing RHEL and Linux Graphical Environment
- 2. Basic System Commands
- 3. Essential File Management Tools
- 4. Working with Users, Groups, and Permissions
- 5. Managing Software
- 6. Managing Processes

Lab safety: Practise on your own RHEL virtual machine. Take a VM snapshot before system-level changes. Check file paths, disk names and process IDs before running commands that modify or delete anything.

Command formatting: every command is shown in bold in the left column; its purpose is explained in one line in the right column. Replace example names such as username, package-name and PID with real values from your lab system.

Practical 1: Installing Red Hat Enterprise Linux (VirtualBox/VMware) and Linux Graphical Environment

Follow the installation wizard in VirtualBox or VMware to create a VM, attach the RHEL ISO, allocate suitable memory and disk space, install RHEL, create a user, and reboot. The commands below are for checking the installed system.

1(a) Installing RHEL in VirtualBox / VMware

COMMAND	ONE-LINE DESCRIPTION
`cat /etc/redhat-release`	Displays the installed Red Hat Enterprise Linux release.
`uname -r`	Displays the currently running Linux kernel version.
`hostnamectl`	Shows hostname, operating system and hardware information.
`lsblk`	Lists disks and partitions so you can inspect the VM's storage devices.
`ip addr`	Displays available network interfaces and their IP addresses.

1(b)(i) Display / Login Manager

COMMAND	ONE-LINE DESCRIPTION
`systemctl status display-manager`	Checks the status of the system's configured display manager.
`systemctl status gdm`	Checks GNOME Display Manager status on a typical RHEL GNOME installation.
`systemctl get-default`	Shows the system's default boot target.

1(b)(ii) Desktop Environment

COMMAND	ONE-LINE DESCRIPTION
`echo $XDG_SESSION_TYPE`	Displays whether the current desktop session uses Wayland or X11.
`echo $XDG_CURRENT_DESKTOP`	Displays the desktop environment name for the current session.
`loginctl show-session "$XDG_SESSION_ID" -p Type -p Desktop`	Shows session type and desktop details for the current login.

1(b)(iii) Linux Directory Structure and File Systems

COMMAND	ONE-LINE DESCRIPTION
`ls -l /`	Lists the top-level Linux directories.
`ls -ld /home /root /etc /var /usr /tmp /boot`	Displays details of common system directories.
`df -hT`	Shows mounted filesystems, their types and human-readable disk usage.
`findmnt`	Displays the mounted filesystem hierarchy.
`mount`	Lists currently mounted filesystems.
`lsblk -f`	Shows block devices, filesystem types, labels and UUIDs.

Optional target commands: systemctl set-default graphical.target sets graphical mode as the default for future boots; systemctl set-default multi-user.target sets text/multi-user mode. Changing the default target affects the next boot.

Practical 2: Basic System Commands

The subtopics below follow Practical 2(a)–(d) in the college index.

2(a) Use input-output redirection (>, >>, |, 2>, etc.)

COMMAND	ONE-LINE DESCRIPTION
`whoami > user.txt`	Writes the current username to user.txt, replacing its previous contents.
`date >> user.txt`	Appends the current date and time to user.txt.
`ls /missing-path 2> error.txt`	Redirects error messages to error.txt.
`ls /etc /missing-path > result.txt 2>&1`	Redirects standard output and error output to the same file.
`wc -l < user.txt`	Uses user.txt as input and counts its lines.
`date | tee date.txt`	Displays the date and saves a copy of the output in date.txt.

2(b) Use grep and regular expressions to analyse text

COMMAND	ONE-LINE DESCRIPTION
`grep "root" /etc/passwd`	Finds lines containing the word root in /etc/passwd.
`grep -i "error" logfile.txt`	Searches for error without distinguishing uppercase and lowercase.
`grep -n "root" /etc/passwd`	Displays matching lines with their line numbers.
`grep -R "PermitRootLogin" /etc/ssh`	Searches recursively for the SSH setting in the directory.
`grep -E "colou?r" words.txt`	Uses an extended regular expression to match colour or color.
`grep "^root" /etc/passwd`	Finds lines that begin with root.
`grep "bash$" /etc/passwd`	Finds lines that end with bash.

2(c) Log in and switch users in multi-user targets

COMMAND	ONE-LINE DESCRIPTION
`whoami`	Displays the username of the current session.
`who`	Lists users currently logged in.
`w`	Shows logged-in users and what they are doing.
`id`	Displays the current user's UID, GID and group memberships.
`su - username`	Switches to another user with that user's login environment.
`exit`	Closes the current shell or returns to the previous user.
`sudo command`	Runs the specified command with configured administrative privileges.
`systemctl get-default`	Displays the default systemd target (boot mode).
`systemctl list-units --type=target`	Lists active systemd target units.

2(d) Working with Text Files: cat, cut, less, head, tail, sort, wc, grep, awk

COMMAND	ONE-LINE DESCRIPTION
`cat file.txt`	Displays the complete contents of a text file.
`cat -n file.txt`	Displays a file with line numbers.
`cut -d',' -f1,3 data.csv`	Extracts fields 1 and 3 from a comma-separated file.
`less file.txt`	Opens a file page by page; press q to quit.
`head file.txt`	Displays the first 10 lines of a file by default.
`tail file.txt`	Displays the last 10 lines of a file by default.

COMMAND	ONE-LINE DESCRIPTION
`tail -f /var/log/messages`	Continuously displays new lines added to the log file.
`sort names.txt`	Sorts lines alphabetically.
`sort -n numbers.txt`	Sorts lines according to numeric values.
`wc file.txt`	Counts lines, words and bytes in a file.
`wc -l file.txt`	Counts the number of lines in a file.
`grep "word" file.txt`	Displays lines containing the specified word.
`awk '{print $1}' file.txt`	Prints the first whitespace-separated field from each line.
`awk -F: '{print $1}' /etc/passwd`	Uses colon as the separator and prints the first field.

Practical 3: Essential File Management Tools

Use a separate practice directory for file and permission exercises. The subheadings follow Practical 3(a)–(l) in the college index.

3(a) Managing Files, Using Links

COMMAND	ONE-LINE DESCRIPTION
`mkdir -p /root/linux-practice`	Creates the practice directory and any missing parent directories.
`cd /root/linux-practice`	Changes the current working directory to the practice directory.
`touch file1.txt`	Creates an empty file or updates its timestamp.
`ls -la`	Lists all files, including hidden files, with details.

3(b) Working with Archives and Compressed Files

COMMAND	ONE-LINE DESCRIPTION
`tar -cvf backup.tar documents/`	Creates an uncompressed tar archive of the documents directory.
`tar -tvf backup.tar`	Lists the contents of a tar archive.
`tar -xvf backup.tar`	Extracts files from a tar archive.
`tar -czvf backup.tar.gz documents/`	Creates a gzip-compressed tar archive.
`tar -xzvf backup.tar.gz`	Extracts a gzip-compressed tar archive.
`tar -cjvf backup.tar.bz2 documents/`	Creates a bzip2-compressed tar archive.
`tar -xjvf backup.tar.bz2`	Extracts a bzip2-compressed tar archive.

3(c) Create, delete, copy, and move files and directories

COMMAND	ONE-LINE DESCRIPTION
`mkdir documents`	Creates a directory named documents.
`cp file1.txt documents/`	Copies file1.txt into the documents directory.
`mv file1.txt notes.txt`	Renames or moves a file.
`rm notes.txt`	Deletes the specified file.
`rmdir documents`	Deletes an empty directory.
`rm -r documents`	Deletes a directory and its contents recursively; use with extreme care.

Before using rm or rm -r, confirm the current location with pwd and check the target with ls. Deleted files may not be recoverable.

3(d) Archive, compress, unpack, and uncompress using tar, star, gzip, and bzip2

COMMAND	ONE-LINE DESCRIPTION
`gzip file.txt`	Compresses file.txt and normally replaces it with file.txt.gz.
`gunzip file.txt.gz`	Decompresses a gzip file.
`bzip2 file.txt`	Compresses file.txt and normally replaces it with file.txt.bz2.
`bunzip2 file.txt.bz2`	Decompresses a bzip2 file.
`star -c -f=backup.star documents/`	Creates a star archive, if the star utility is installed.
`star -t -f=backup.star`	Lists the contents of a star archive.
`star -x -f=backup.star`	Extracts a star archive.

The star utility may not be installed by default. Check with command -v star. Use tar for the standard archive exercises if star is unavailable.

3(e) Create hard and soft links

COMMAND	ONE-LINE DESCRIPTION
`ln original.txt hardlink.txt`	Creates a hard link to the same inode as original.txt.
`ln -s original.txt softlink.txt`	Creates a symbolic (soft) link pointing to original.txt.
`ls -li`	Displays inode numbers, helping distinguish hard links and symbolic links.
`stat original.txt hardlink.txt softlink.txt`	Displays detailed file and link information.

3(f) Managing File Ownership, Managing Basic Permissions

COMMAND	ONE-LINE DESCRIPTION
`chown username file.txt`	Changes the owner of file.txt.
`chown username:groupname file.txt`	Changes both the owner and group of file.txt.
`chgrp groupname file.txt`	Changes the group ownership of file.txt.
`chmod 755 script.sh`	Sets owner permissions to rwx and group/other permissions to r-x.
`chmod 644 file.txt`	Sets owner permissions to rw- and group/other permissions to r--.
`chmod u+x script.sh`	Adds execute permission for the file owner.
`chmod g-w file.txt`	Removes write permission from the file's group.
`ls -l`	Displays file permissions and ownership.

3(g) Managing Advanced Permissions, Managing ACLs

COMMAND	ONE-LINE DESCRIPTION
`chmod u+s program`	Sets the SUID permission bit on a suitable executable.
`chmod g+s shared-directory`	Sets the SGID bit on a directory so new files inherit its group.
`chmod +t shared-directory`	Sets the sticky bit on a shared directory.
`getfacl file.txt`	Displays the file's access control list.
`setfacl -m u:username:rw file.txt`	Grants a named user read and write ACL permissions.
`setfacl -m g:groupname:r file.txt`	Grants a named group read ACL permission.
`setfacl -x u:username file.txt`	Removes the named user's ACL entry.
`setfacl -b file.txt`	Removes extended ACL entries from the file.

3(h) Setting Default Permissions with umask

COMMAND	ONE-LINE DESCRIPTION
`umask`	Displays the current file-creation permission mask.
`umask 022`	Sets the permission mask to 022 for the current shell.
`umask -S`	Displays the mask in symbolic form on supported shells.

3(i) Working with User-Extended Attributes

COMMAND	ONE-LINE DESCRIPTION
`getfattr -d file.txt`	Displays extended attributes attached to the file.
`setfattr -n user.note -v "lab file" file.txt`	Creates a user.note extended attribute with the value lab file.
`getfattr -n user.note file.txt`	Displays the value of the named extended attribute.
`setfattr -x user.note file.txt`	Removes the named extended attribute.

3(j) List, set, and change standard UGO/RWX permissions

COMMAND	ONE-LINE DESCRIPTION
`ls -l file.txt`	Lists the file's UGO permissions in symbolic form.
`chmod 640 file.txt`	Sets owner rw-, group r-- and others no permissions.
`chmod u=rw,g=r,o= file.txt`	Sets permissions symbolically for user, group and others.

UGO means User, Group and Others. RWX means Read, Write and Execute. Numeric values are read=4, write=2 and execute=1.

3(k) Create and configure set-GID directories for collaboration

COMMAND	ONE-LINE DESCRIPTION
`mkdir shared`	Creates a directory for group collaboration.
`chgrp labgroup shared`	Assigns the shared directory to the labgroup group.
`chmod 2775 shared`	Sets SGID and grants owner/group full access and others read/execute access.
`ls -ld shared`	Checks the directory's group ownership and SGID permission.

3(l) Create and manage access control lists

COMMAND	ONE-LINE DESCRIPTION
`setfacl -m u:student1:rwx shared`	Grants student1 read, write and execute permissions on shared.
`setfacl -m d:u:student1:rwx shared`	Sets a default ACL for student1 on newly created items in shared.
`getfacl shared`	Displays the directory ACL and its default ACL entries.
`setfacl -x u:student1 shared`	Removes student1's named ACL entry.

Practical 4: Working with Users, Groups, and Permissions

Run user and group administration commands with root privileges or through sudo. Use disposable lab accounts.

4(a) Creating and Managing Users

COMMAND	ONE-LINE DESCRIPTION
`useradd student1`	Creates a local user account named student1.
`passwd student1`	Sets or changes the password for student1.
`id student1`	Displays the user's UID, primary GID and group memberships.
`getent passwd student1`	Displays the local account database entry for student1.
`usermod -s /bin/bash student1`	Changes the user's login shell to Bash.
`usermod -aG wheel student1`	Adds student1 to the wheel group without replacing existing supplementary groups.
`usermod -L student1`	Locks the user's password authentication.
`usermod -U student1`	Unlocks the user's password authentication.

4(b) Creating and Managing Groups

COMMAND	ONE-LINE DESCRIPTION
`groupadd labgroup`	Creates a group named labgroup.
`getent group labgroup`	Displays the group database entry.
`usermod -aG labgroup student1`	Adds student1 to labgroup as a supplementary group.
`groupmod -n newlabgroup labgroup`	Renames labgroup to newlabgroup.
`groupdel newlabgroup`	Deletes the specified group if it is no longer in use.

4(c) Using Permissions and Advanced Permissions

COMMAND	ONE-LINE DESCRIPTION
`chmod 750 project.sh`	Sets owner rwx, group r-x and no permissions for others.
`chmod g+s shared`	Sets SGID on the shared directory.
`chmod +t shared`	Sets the sticky bit on the shared directory.
`chown student1:labgroup project.sh`	Sets the file owner and group.
`getfacl project.sh`	Displays ACL entries and effective permissions.

4(d) Change passwords and adjust password aging for local user accounts

COMMAND	ONE-LINE DESCRIPTION
`chage -l student1`	Displays password-aging details for student1.
`chage -d 0 student1`	Forces student1 to change the password at next login.
`chage -M 90 student1`	Sets the maximum password age to 90 days.
`chage -m 1 student1`	Sets the minimum password age to one day.
`chage -W 7 student1`	Sets a seven-day warning period before password expiry.

4(e) Configure superuser access: su, sudo, root

COMMAND	ONE-LINE DESCRIPTION
`su -`	Switches to the root account with a root login environment.
`exit`	Leaves the root shell and returns to the previous user.

COMMAND	ONE-LINE DESCRIPTION
`sudo -l`	Lists commands the current user is allowed to run with sudo.
`sudo useradd student2`	Runs useradd with administrative privileges.
`visudo`	Safely edits sudoers configuration while checking its syntax.

Use userdel or userdel -r only for disposable practice accounts. The -r option also removes the user's home directory and mail spool. Do not remove real user accounts.

Practical 5: Managing Software

The college index specifies YUM, RPM and installing/updating packages from a local filesystem repository. On RHEL 8/9, yum is DNF-compatible; dnf commands are also provided.

5(a) Managing Software Packages with YUM

COMMAND	ONE-LINE DESCRIPTION
`yum repolist`	Lists enabled YUM repositories.
`yum search package-name`	Searches enabled repositories for a package.
`yum info package-name`	Displays package information.
`yum install package-name`	Installs a package and its dependencies from enabled repositories.
`yum update package-name`	Updates the specified installed package.
`yum remove package-name`	Removes the specified package.

5(b) Using yum

COMMAND	ONE-LINE DESCRIPTION
`dnf repolist`	Lists enabled DNF/YUM-compatible repositories.
`dnf makecache`	Downloads and builds repository metadata cache.
`dnf search httpd`	Searches repositories for packages matching httpd.
`dnf info httpd`	Displays information about the httpd package.
`dnf list installed`	Lists installed packages.
`dnf update`	Updates installed packages from enabled repositories.
`rpm -q httpd`	Checks whether the httpd package is installed.

5(c) Managing Software Packages with RPM

COMMAND	ONE-LINE DESCRIPTION
`rpm -ivh package.rpm`	Installs a local RPM package and shows progress.
`rpm -Uvh package.rpm`	Upgrades or installs a local RPM package.
`rpm -q package-name`	Checks whether a package is installed.
`rpm -qa`	Lists all installed RPM packages.
`rpm -ql package-name`	Lists files installed by a package.
`rpm -qf /usr/bin/ls`	Identifies which installed package owns a file.
`rpm -V package-name`	Verifies installed files against RPM database metadata.
`dnf install ./package.rpm`	Installs a local RPM and can resolve dependencies from enabled repositories.

RPM alone does not automatically resolve dependencies. For local RPM installation, dnf install ./package.rpm is generally preferable when repository access is available.

5(d) Install and update software packages from the local file system repository

For an offline VirtualBox/VMware lab, attach the matching RHEL ISO to the virtual CD/DVD drive before mounting it.

COMMAND	ONE-LINE DESCRIPTION
`lsblk`	Checks whether the virtual optical drive and other block devices are detected.
`mkdir -p /mnt/cdrom`	Creates a mount point for the installation media.
`mount /dev/sr0 /mnt/cdrom`	Mounts the ISO/DVD if /dev/sr0 is the correct optical device.

COMMAND	ONE-LINE DESCRIPTION
`ls /mnt/cdrom`	Checks the contents of the mounted ISO.
`cat > /etc/yum.repos.d/local.repo <<'EOF' ... EOF`	Creates a local repository configuration file for BaseOS and AppStream.
`dnf clean all`	Clears cached repository metadata.
`dnf makecache`	Builds metadata cache from the configured local repositories.
`dnf repolist`	Verifies that the local repositories are enabled.
`dnf install package-name`	Installs a package from the configured local repository.
`cd / && umount /mnt/cdrom`	Moves out of the mount directory and unmounts the ISO.

Contents of /etc/yum.repos.d/local.repo:

```
[BaseOS]
name=RHEL Local BaseOS
baseurl=file:///mnt/cdrom/BaseOS
enabled=1
gpgcheck=0

[AppStream]
name=RHEL Local AppStream
baseurl=file:///mnt/cdrom/AppStream
enabled=1
gpgcheck=0
```

If mount reports 'no medium found on /dev/sr0', the ISO is usually not attached to the VM's virtual optical drive. Attach the correct RHEL ISO in VirtualBox/VMware Storage settings and retry. The ISO should match the installed RHEL major version and architecture.

Practical 6: Managing Processes

The subtopics below follow Practical 6(a)–(f) in the college index.

6(a) Introduction to Process Management

COMMAND	ONE-LINE DESCRIPTION
`ps`	Displays processes associated with the current terminal.
`ps -ef`	Displays a full-format list of running processes.
`pstree`	Displays processes in a parent-child tree.
`uptime`	Shows system uptime and load averages.
`free -h`	Displays memory and swap usage in human-readable units.

6(b) Managing Shell Jobs

COMMAND	ONE-LINE DESCRIPTION
`sleep 300`	Starts a sample foreground process for practising job control.
`jobs`	Lists jobs started by the current shell.
`bg`	Resumes a suspended job in the background.
`fg`	Brings a background or suspended job to the foreground.
`sleep 300 &`	Starts the sample sleep process in the background.
`nohup command > output.log 2>&1 &`	Runs a command so it can continue after logout, saving output to a log.

To practise suspension, start sleep 300, press Ctrl+Z, then use jobs, bg and fg. Press Ctrl+C to interrupt a foreground command.

6(c) Using Common Command-Line Tools for Process Management

COMMAND	ONE-LINE DESCRIPTION
`pgrep -a process-name`	Finds process IDs and command lines matching a process name.
`pidof process-name`	Displays process IDs for a named program.
`kill -l`	Lists available signal names and numbers.
`kill PID`	Sends the default SIGTERM signal to the selected process.
`kill -15 PID`	Requests normal termination using SIGTERM.
`pkill process-name`	Sends the default termination signal to matching processes; verify the match first.
`nice -n 10 command`	Starts a command with a lower scheduling priority.
`renice 10 -p PID`	Changes the nice value of an existing process.
`lsof -p PID`	Lists files opened by the selected process.

6(d) Using top to Manage Processes

COMMAND	ONE-LINE DESCRIPTION
`top`	Displays live process, CPU and memory usage; press q to quit.
`top -u username`	Displays processes belonging to the specified user.

6(e) Using tuned to Optimize Performance

COMMAND	ONE-LINE DESCRIPTION
`systemctl status tuned`	Checks whether the tuned service is running.
`systemctl enable --now tuned`	Enables and starts the tuned service.

COMMAND	ONE-LINE DESCRIPTION
`tuned-adm list`	Lists available tuned performance profiles.
`tuned-adm active`	Displays the currently active tuned profile.
`tuned-adm recommend`	Suggests a tuned profile for the system.
`tuned-adm profile balanced`	Activates the balanced performance profile.

6(f) Identify CPU/memory-intensive processes and kill processes

COMMAND	ONE-LINE DESCRIPTION
`ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | head`	Shows processes using the most CPU at the top.
`ps -eo pid,ppid,comm,%mem,%cpu --sort=-%mem | head`	Shows processes using the most memory at the top.
`free -h`	Checks overall memory availability and usage.
`vmstat 1 5`	Reports memory, CPU and system activity at one-second intervals for five reports.
`kill PID`	Requests a process to terminate normally.
`kill -9 PID`	Forces process termination; use only as a last resort.

Always verify the process ID and command name before terminating anything. Prefer kill PID (SIGTERM) first. Use kill -9 only if the process does not stop normally. Do not terminate unfamiliar system processes.

Index Coverage Checklist

- Practical 1 — RHEL installation; display/login manager; desktop environment; Linux directory structure and filesystems.
- Practical 2 — input/output redirection; grep and regular expressions; switching users and multi-user targets; text-file commands.
- Practical 3 — files and links; archives/compression; file operations; tar/star/gzip/bzip2; ownership; basic/advanced permissions; umask; extended attributes; UGO/RWX; SGID; ACL.
- Practical 4 — users; groups; permissions and advanced permissions; passwords and password aging; su/sudo/root.
- Practical 5 — YUM; yum usage; RPM; local filesystem repository installation and updates.
- Practical 6 — process management; shell jobs; CLI process tools; top; tuned; CPU/memory-intensive processes and termination.

Package availability and command options can vary with RHEL version and enabled repositories. Some utilities such as star, getfattr/setfattr and lsof may need to be installed separately. Commands that change system configuration should be run with appropriate administrative privileges.

Prepared as a lab reference for RHEL 8 / RHEL 9. The practical sequence and subtopic grouping follow the supplied college index for Practicals 1–6.